

# End-to-End Helm Deployment Guide for Moodle on Kubernetes

## End-to-End Helm Deployment Guide for Moodle on Kubernetes

This document provides a **complete, practical walkthrough** of deploying **Moodle LMS** using **Helm**, compares **Helm vs Kustomize**, explains **Helm integration with open-source ITSM / Change Management**, and illustrates how **Azure DevOps → Helm → AKS** works together.

---

### 1. Full Moodle Deployment Using Helm (Step by Step)

#### 1.1 What Moodle Needs in Kubernetes

A production-grade Moodle setup requires:

- Moodle application pods
- A database (MariaDB / MySQL / PostgreSQL)
- Persistent storage for:
  - `moodledata`
  - database data

- Secrets for credentials
- Services and optional Ingress
- Support for upgrades and rollbacks

Managing this manually with raw YAML easily grows to **10+ files**.

Helm packages all of this into a **single deployable unit**.

---

## 1.2 Choosing a Helm Chart

A widely used and production-ready option is the **Bitnami Moodle Helm chart**.

What the chart includes:

- Moodle Deployment, Service, PVCs
  - Database sub-chart (MariaDB)
  - Secrets and ConfigMaps
  - Ingress support
  - Upgrade and rollback capability
- 

## 1.3 Install Helm

Shell



```
1 helm version
2
```

Helm v3 is required.

---

## 1.4 Add the Helm Repository

Shell



```
1 helm repo add bitnami https://charts.bitnami.com/bitnami
2 helm repo update
3
```

## 1.5 Create Environment-Specific Values File

Example: **values-prod.yaml**

YAML



```
1 replicaCount: 2
2
3 image:
```

```
4   tag: 4.4
5
6   moodle:
7     username: admin
8     password: strongpassword
9     email: admin@example.com
10
11  mariadb:
12    enabled: true
13    auth:
14      rootPassword: rootpassword
15      database: moodle
16
17  persistence:
18    enabled: true
19    size: 50Gi
20
21  ingress:
22    enabled: true
23    hostname: moodle.example.com
24
```

✅ This single file controls the entire deployment

## 1.6 Deploy Moodle

Shell

```
1 helm install moodle-prod bitnami/moodle \
2   --namespace lms-prod \
3   --create-namespace \
4   --values values-prod.yaml
```

Helm will:

- Create all Kubernetes objects
  - Set up storage and database
  - Track this deployment as a **release**
- 

## 1.7 Upgrade Moodle

Change the image version:

```
1 image:
2   tag: 4.5
3
```

YAML



Upgrade:

```
1 helm upgrade moodle-prod bitnami/moodle \
2   --values values-prod.yaml
3
```

Shell



## 1.8 Rollback Moodle

Shell 

```
1 helm rollback moodle-prod 1
2
```

- ✓ Restores the previous known-good version
  - ✓ No raw YAML redeployments needed
- 

## 2. Helm vs Kustomize (Clear Comparison)

### 2.1 What Helm Is

- A **package manager** for Kubernetes
  - Uses templates + values
  - Maintains release history
  - Supports upgrades and rollbacks
-

## 2.2 What Kustomize Is

- A **YAML customization tool**
  - No templating language
  - Overlays modify base YAML
  - No release tracking or rollback
- 

## 2.3 Key Differences

|   | ☰ Feature           | ☰ Helm             | ☰ Kustomize   |
|---|---------------------|--------------------|---------------|
| 1 | Templating          | Yes (Go templates) | No            |
| 2 | Package concept     | Yes (Charts)       | No            |
| 3 | Release history     | Yes                | No            |
| 4 | Rollback            | Built-in           | Manual        |
| 5 | App store ecosystem | Yes                | No            |
| 6 | Best for            | Complex apps       | Config tweaks |

---

## 2.4 Which Should You Use?

-  **Helm:** Moodle, CMS, stateful apps, CI/CD pipelines
-  **Kustomize:** Simple overlays, environment differences, GitOps adjustments

Many teams use **both**:

- Helm to install apps
  - Kustomize to adapt cluster-specific settings
- 

## 3. Helm + Open-Source ITSM Change Management Integration

### 3.1 Why Integrate Change Management?

In enterprise environments:

- Production changes require approval
  - Deployments must be auditable
  - Rollbacks must be tracked
-



## 3.2 Open-Source ITSM Options

Common systems:

- iTop
- GLPI
- OTRS / Znuny

Helm integrates indirectly via **CI/CD pipelines + REST APIs**.

---

## 3.3 Example Flow

1. Azure DevOps pipeline starts
  2. Pipeline creates a Change Request via REST API
  3. Change ID stored as pipeline variable
  4. Approval gate validates change status
  5. Helm deploys Moodle
  6. Pipeline updates Change status to Implemented
- 

## 3.4 Example Conceptual Pipeline Step

Shell

```
1 curl -X POST https://itsm/api/changes \
```

```
2   -d '{"title":"Moodle upgrade","version":"4.5"}'  
3
```

✅ Helm acts as the **change executor** ✅ ITSM acts as the **approval authority**

---

## 4. Azure DevOps → Helm → AKS Architecture Flow

### 4.1 Logical Flow

1 Developer  
2 ↓  
3 Azure DevOps Repo  
4 ↓  
5 Azure DevOps Pipeline  
6 ↓  
7 Helm Upgrade / Install  
8 ↓  
9 AKS API Server  
10 ↓  
11 Kubernetes Resources Created  
12 ↓  
13 Running Moodle Pods  
14

Plain Text



## 4.2 Pipeline Responsibilities

|   | ≡ Stage  | ≡ Responsibility         |
|---|----------|--------------------------|
| 1 | Build    | Validate config & values |
| 2 | Approval | ITSM Change approval     |
| 3 | Deploy   | Helm install/upgrade     |
| 4 | Verify   | Health & smoke tests     |
| 5 | Close    | Update Change status     |

## 4.3 Why This Works Well

-  Single source of truth (Helm values)
-  Repeatable, auditable deployments
-  Automatic rollback capability
-  Clean separation of concerns

## 5. Final Takeaway

Helm transforms complex applications like Moodle into versioned, repeatable, auditable deployments that integrate cleanly with Azure DevOps, AKS, and enterprise Change Management systems.

---

If you want next:

- A real Azure DevOps YAML pipeline example
- A Helm vs raw YAML visual diff
- A GitOps (ArgoCD) version of this flow

Just tell me.